

COGNDB // GRAPH ARCHITECTURE & SUPPLY CHAIN LEDGER

DEPENDENCY & VULNERABILITY EXPLORER

Comprehensive Engineering Blueprint, Standout Innovations & Submission Dossier

SYSTEM SPECIFICATIONS & CODE QUALITY

- Engine: CognoDB Cloud (Neo4j Bolt 5.0-5.4)
- Graph Protocol: openCypher with Index-Free Adjacency
- Backend: Node.js 20, Express, Neo4j Driver Connection Pool
- Frontend: React 18, Monotone Ink System, 60 FPS Canvas
- Dataset: 75 Packages, 77 Versions, 10 Maintainers, 17 CVEs

- Innovations: In-Browser Cypher IDE, What-If Sandbox, Patch Engine

- Code Quality: 100/100 React Doctor Score (0 Warnings)

- Responsive: Mobile, Tablet, Laptop, Ultra-Wide (4K Ready)

WEXA AI // TAKE-HOME ASSIGNMENT 2 // SUBMISSION DOSSIER
Target Evaluation: 100% Graph-Native Architecture & Production Quality

Table of Contents

COMPREHENSIVE MASTER NAVIGATION

Section	Description	Location
1. Executive Summary & Graph Paradigm	Why software supply chains require graph databases over SQL CTEs	Page 3
2. Graph Data Model & Schema	5 Node Labels, 5 Relationships, properties & seed dataset audit	Page 4
3. Cypher Query Masterclass	Multi-hop traversals, shortestPath(), and maintainer centrality	Page 5
4. Backend Architecture & Resilience	Connection pooling, keep-alive recycling, retry wrappers & fallbacks	Page 7
5. Frontend & Monotone Ink System	Canvas physics, Golden Spiral layout, Lucide icons & Ctrl+K	Page 8
6. Feature & Screen Walkthrough	Dashboard, Package Dossier, Vulnerability Hub & Global Topology	Page 10
7. Standout Innovations Deployed	In-Browser Cypher IDE, What-If Sandbox, Patch Recommender, SBOM	Page 12
8. React Doctor & Code Health (100/100)	Rules resolved, useReducer state machines & giant component fixes	Page 14
9. Responsive Design & Pagination	Fluid layouts, mobile bottom nav, centered 10-100 pagination	Page 15
10. GitHub Repo & Setup Protocol	Repository structure, environment config, seed script & run commands	Page 16
11. Interview Defense & Video Script	Top evaluator technical questions, exact answers & demo script	Page 17

1. Executive Summary & Graph Paradigm

FUNDAMENTAL PRINCIPLES

1.1 The Core Problem in Open-Source Supply Chains

Modern software engineering relies heavily on open-source packages. When an enterprise application declares 10 direct dependencies, those dependencies pull in hundreds of transitive packages across 3 to 6 tiers.

When a critical vulnerability (such as CVE-2025-10234) is disclosed in a deeply nested utility package (e.g. encode-utls), security teams face two critical questions that determine their attack exposure:

- **Transitive Exposure:** Is our core application exposed to this flaw, even if it sits 4 hops deep?
- **Maintainer Leverage:** Which open-source authors have the largest downstream blast radius if their accounts were hijacked?

1.2 Connections vs. Rows: Why Graph Databases Outperform Relational SQL

Relational databases organize information into tabular rows and foreign keys. Answering multi-hop dependency questions in SQL requires recursive Common Table Expressions (CTEs) or iterative self-joins. As traversal depth increases past 2 hops, relational engines suffer from exponential combinatorial join explosion, resulting in memory exhaustion and severe latency.

CognoDB (an openCypher graph engine) utilizes Index-Free Adjacency. Relationships are stored as direct physical memory pointers. Traversing from a package to its dependencies is an $O(1)$ pointer dereference rather than an $O(\log N)$ B-Tree index lookup. Graph traversal execution time is proportional only to the size of the subgraph explored ($O(V + E)$), completely independent of the total size of the global database.

1.3 Relational SQL CTE vs. CognoDB Cypher Comparison

Below is the dramatic difference in code complexity and maintainability between relational SQL CTEs and declarative Cypher:

Relational SQL Recursive CTE (Complex, Verbose, Fragile):

```

WITH RECURSIVE dependency_chain AS (
  SELECT root_pkg_id, dep_pkg_id, 1 AS depth, ARRAY[root_pkg_id, dep_pkg_id] AS path
  FROM package_dependencies WHERE root_pkg_id = 'enterprise-api-gateway'
  UNION ALL
  SELECT dc.root_pkg_id, pd.dep_pkg_id, dc.depth + 1, dc.path || pd.dep_pkg_id
  FROM dependency_chain dc
  JOIN package_dependencies pd ON dc.dep_pkg_id = pd.pkg_id
  WHERE dc.depth < 6 AND NOT pd.dep_pkg_id = ANY(dc.path) -- Manual cycle detection
)
SELECT dc.path, v.cve_id, v.severity, dc.depth
FROM dependency_chain dc
JOIN package_versions pv ON dc.dep_pkg_id = pv.pkg_id
JOIN version_vulnerabilities vv ON pv.version_id = vv.version_id
JOIN vulnerabilities v ON vv.cve_id = v.cve_id ORDER BY dc.depth ASC LIMIT 10;

```

CognoDB Cypher Variable-Length Traversal (Clean, Declarative, Graph-Native):

```

MATCH (root:Package {name: $rootName})-[:HAS_VERSION]->(v:Version)
MATCH (vuln:Vulnerability)
MATCH path = shortestPath(
  (v)-[:DEPENDS_ON*0..6]->(:Package)-[:HAS_VERSION]->(:Version)-[:AFFECTED_BY]->(vuln)
)
RETURN [n IN nodes(path) WHERE n:Package | n.name] AS path,
       vuln.cveId AS cveId, vuln.severity AS severity, length(path) AS hops
ORDER BY hops ASC LIMIT 10;

```

2. Graph Data Model & Schema Architecture

ENTITY-RELATIONSHIP BLUEPRINT

2.1 Node Entities, Labels & Property Definitions

The graph schema is built around 5 distinct labeled node types, modeling real software supply chains:

Node Label	Key Properties	Semantic Purpose
:Package	name (unique), ecosystem	Represents a software package across
:Version	version, releaseDate	npm, PyPI, or crates. A specific published SemVer revision of a
:Maintainer	name, email (unique)	package. Open-source developer or organization
:License	name, type	publishing versions. Governing legal software license (MIT,
:Vulnerability	cveId (unique), severity, desc	Apache-2.0, etc.). Documented security flaw advisory with CVE severity ranking.

2.2 Relationship Types, Directionality & Semantics

Relationship	Direction	Cardinality	Meaning
HAS_VERSION	(:Package) !' (:Version)	1 : N	Binds a package to its
DEPENDS_ON	(:Version) !' (:Package)	N : M	published releases. Recursive multi-hop edge with
MAINTAINED_BY	(:Version) !' (:Maintainer)	N : 1	{versionRange}. Identifies author responsibility
LICENSED_UNDER	(:Version) !' (:License)	N : 1	per release. Specifies legal compliance and
AFFECTED_BY	(:Version) !' (:Vulnerability)	N : M	governance. Maps CVE security vulnerabilities to versions.

2.3 Seed Dataset Metrics & Distribution

The dataset loaded into
Cognodb Cloud is
specifically engineered with
realistic multi-tiered
vulnerability chains:

- **Total Packages:** 75 real packages (40 npm, 20 PyPI, 15 crates).
- **Published Versions:** 77 semantic version nodes with release metadata.
- **Open-Source Maintainers:** 10 core developers with varied downstream leverage.

- **Indexed CVE Flaws:** 17 active CVEs (6 Critical, 7 High, 4 Medium).
- **Graph Dependency**
Edges: 153 directed relationships spanning up to 4 transitive hops.

3. Cypher Query Masterclass & Complexity

ALGORITHMIC GRAPH TRAVERSAL

3.1 Query 1: Transitive Dependents (Upstream Blast Radius)

This query computes the entire set of upstream applications that rely on a target library across 1 to 4 hops:

```
MATCH path = (dependent:Package)-[:HAS_VERSION]->(:Version)-[:DEPENDS_ON*1..4]->(p:Package {name:$name})
WITH dependent, min(length(path)) AS hops
RETURN DISTINCT dependent.name AS name, dependent.ecosystem AS ecosystem, hops
ORDER BY hops ASC, dependent.name ASC;
```

3.2 Query 2: Variable-Depth Shortest Attack Path

Computes the exact hop-by-hop exploitation chain from an enterprise application down to reachable CVEs:

```
MATCH (root:Package {name: $rootName})-[:HAS_VERSION]->(v:Version)
MATCH (vuln:Vulnerability)
MATCH path = shortestPath(
  (v)-[:DEPENDS_ON*0..6]->(:Package)-[:HAS_VERSION]->(:Version)-[:AFFECTED_BY]->(vuln)
)
RETURN [n IN nodes(path) WHERE n:Package | n.name] AS path,
       vuln.cveId AS cveId, vuln.severity AS severity, vuln.description AS description,
       length(path) AS hops
ORDER BY hops ASC LIMIT 10;
```

3.3 Query 3: Maintainer Blast Radius (Centrality Analysis)

Ranks maintainers by their total downstream impact across the global dependency network:

```
MATCH (m:Maintainer)<-[:MAINTAINED_BY]-(:Version)<-[:HAS_VERSION]-(:Package)
OPTIONAL MATCH (downstream:Package)-[:HAS_VERSION]->(:Version)-[:DEPENDS_ON*1..4]->(p)
WITH m, count(DISTINCT p) AS directPackages, count(DISTINCT downstream) AS downstreamPackages,
collect(DISTINCT p.name) AS packageNames
RETURN m.name AS maintainer, m.email AS email, directPackages,
       downstreamPackages AS packagesAffected, (directPackages + downstreamPackages) AS totalReach,
       packageNames AS packages
ORDER BY packagesAffected DESC, directPackages DESC LIMIT 20;
```

3.4 Query 4: Multi-Subquery Aggregated KPIs

Uses modern Cypher CALL subqueries to aggregate graph metrics in a single network round-trip:


```
CALL { MATCH (p:Package) RETURN count(p) AS packageCount }  
CALL { MATCH (v:Version) RETURN count(v) AS versionCount }  
CALL { MATCH (m:Maintainer) RETURN count(m) AS maintainerCount }  
CALL { MATCH (vuln:Vulnerability) RETURN count(vuln) AS vulnCount }  
CALL { MATCH ()-[r:DEPENDS_ON]->() RETURN count(r) AS dependencyCount }  
RETURN packageCount, versionCount, maintainerCount, vulnCount, dependencyCount;
```

4. Backend Architecture & Resilience

PRODUCTION ENGINEERING BLUEPRINT

4.1 System Overview & Driver Lifecycle

The backend is structured in an Express.js ES-Module architecture. The official neo4j-driver instance is initialized once at startup to manage connection pooling, while each HTTP request creates and destroys an isolated, lightweight session.

4.2 Keep-Alive & Cloud Drop Prevention (Solving ECONNRESET)

Cloud graph databases (like CognoDB Cloud) automatically close idle TCP sockets after 30–60 seconds of inactivity without sending a TCP FIN. To guarantee zero dropped connections, we configured the driver with proactive connection recycling:

```
driver = neo4j.driver(
  COGNODB_URI,
  neo4j.auth.basic(COGNODB_USER, COGNODB_PASSWORD),
  {
    maxConnectionPoolSize: 50,
    maxConnectionLifetime: 30 * 1000,           // 30s connection recycling
    connectionLivenessCheckTimeout: 1000,       // 1s pre-acquisition health ping
    connectionAcquisitionTimeout: 15000,
    connectionTimeout: 15000,
    disableLosslessIntegers: true,
  }
);
```

4.3 Resilient Exponential Backoff Retry Wrapper (withRetry)

All Cypher queries are wrapped in an asynchronous retry harness. If a cloud socket experiences transient latency or reset, the query automatically reconnects and re-executes with exponential backoff.

```
export async function withRetry(fn, retries = 3, delayMs = 400) {
  let lastErr;
  for (let attempt = 1; attempt <= retries; attempt++) {
    try {
      return await fn();
    } catch (err) {
      lastErr = err;
      const isTransient = err.message?.includes('ECONNRESET') || err.code === 'ServiceUnavailable';
      if (attempt < retries && isTransient) {
        await new Promise((res) => setTimeout(res, delayMs * attempt));
      }
    }
  }
  throw lastErr;
}
```

4.4 Zero-Downtime Dual-Engine Architecture

Every route features graceful fallback layers to LocalGraphEngine. If live cloud credentials are absent or during network partition, the application serves the exact same graph dataset and BFS graph traversal algorithms in-memory, ensuring 100% uptime.

5. Frontend Architecture & Design System

MONOTONE INK AESTHETICS

5.1 Monotone Ink Design Philosophy

The user interface is engineered with the Monotone Ink cybersecurity design system matching the Stitch mockups. It features:

- **High-Contrast Layout:** 1px ink borders (#000004) with off-white base (#fbf8ff) and corner accents.
- **Typography Hierarchy:** Source Serif 4 for editorial titles, JetBrains Mono for hashes and CVE tags, Inter for copy.
- **Native Lucide React Icons:** High-definition vector components replacing legacy font icon spans.
- **Responsive App Shell:** Desktop fixed sidebar navigation + mobile bottom navigation dock.

5.2 High-Performance HTML5 Canvas Force-Directed Engine

To render 64 nodes and 116 links at silky-smooth 60 FPS without CPU lag, the visualizer implements:

- **Pure Ref-Based State:** Physics simulation loop reads from mutable useRef objects, preventing React re-render cycles.
- **Archimedean Golden Spiral Layout:** Distributes multi-node networks using $r = 38 * \sqrt{i + 1}$ and $\theta = i * 137.5$ deg.
- **Auto-Fit Viewbox Bounding Box:** Measures [minX, maxX, minY, maxY] and automatically scales/centers the network on mount.
- **Native Scroll Isolation:** Non-passive wheel listener stops window scrolling during canvas zoom.
- **Snapshot Exporter:** Captures crystal-clear high-res PNG snapshots with 1-click.

5.3 Global Quick Command Palette (Ctrl+K)

A native HTML5 dialog command palette allows instant keyboard navigation across all 75 packages and 17 CVE advisories from anywhere in the application.

6. Detailed Feature & Screen User Journey

INTERACTIVE WALKTHROUGH

6.1 Dashboard & Search Registry (Route: /)

- **KPI Metrics Ribbon:** 5 live stat cards aggregating packages, versions, maintainers, CVEs, and dependency edges.
- **Debounced Search Box:** 300ms real-time search with trending query chips.
- **Centered Bottom Pagination:** 10, 20, 50, 100 page size options with smart state reset.

6.2 Package Detail & Dossier (Route: /package/:name)

- **Header & Metadata:** Ecosystem pill, SemVer version, SPDX License, and primary Maintainer.
- **Multi-Format Exporter:** Exports full dossier as JSON Manifest (.json), CSV Spreadsheet (.csv), or Markdown (.md).
- **Automated Remediation Card:** Calculates non-breaking package upgrade paths to sever CVE exposure.
- **4-Tab Workspace:** Interactive Subgraph, Attack Paths, Transitive Dependents, Downstream Dependencies.

6.3 Vulnerability Hub (Route: /vulnerabilities)

- **Blast Radius Leaderboard:** Visual impact bars ranking maintainers by downstream reach percentage.
- **Searchable CVE Catalog:** Search and filter 17 CVEs by severity (Critical, High, Medium).
- **Shortest Attack Path Simulator:** Select any package to calculate exact multi-hop attack propagation steps.

6.4 Global Topology Network (Route: /graph)

- **Ecosystem Network View:** Interactive canvas force graph visualizer with ecosystem filtering.
- **Node Inspector Dossier:** Clicking any node opens a panel showing all connected relationships and direct navigation links.

7. Standout Innovations Deployed

VALUE-ADDED GRAPH FEATURES

7.1 In-Browser openCypher Query Console (Route: /cypher)

A live interactive Cypher IDE embedded directly in the web application:

- **Pre-Loaded Traversal Templates:** Shortest attack paths, maintainer centrality ranking, transitive reach, and system KPIs.
- **Execution Telemetry:** Measures real-time query duration in milliseconds (e.g. 12 ms), target cluster engine, and row count.
- **Keyboard Execution:** Supports Ctrl+Enter instant query execution.
- **Table & JSON Exporters:** 1-click export of custom query results.

7.2 "What-If" Blast Radius Simulation Sandbox (Route: /sandbox)

A security sandbox allowing evaluators to simulate zero-day supply chain strikes:

- **Target Selection:** Select any package or maintainer to simulate quarantine.
- **Disruption Score Gauge:** Computes percentage of global microservices taken down.
- **Cascading Casualties Ledger:** Lists all affected downstream services categorized by propagation depth (Ground Zero, Direct Dependent, Transitive Cascade).

7.3 Automated Vulnerability Remediation Engine

Computes minimal non-breaking SemVer upgrades to resolve multi-hop CVE chains with copy-paste terminal commands.

8. React Doctor & Code Health (100/100)

ZERO-WARNING STANDARDS

8.1 React Doctor Score Progression

The frontend was audited and optimized across multiple refactoring iterations using the official React Doctor tool:

Audit Iteration	Score	Key Improvements Made
Initial Baseline	61 / 100	Identified array index keys, multiple
Pass 1: Key Stability	65 / 100	useStates, and giant components. Resolved react-doctor/no-array-index-as-
Pass 2: useReducer	66 / 100	key across 7 files. Unified 7 interdependent states in
Pass 3: ES2023 Sort	67 / 100	PackagePage into clean reducer actions. Replaced [...arr].sort() with native
Pass 4: Modularization	73 / 100	immutable arr.toSorted(). Decomposed 4 oversized components into
Pass 5: Accessibility	90 / 100	12 subcomponents & custom hooks. Added aria-labels, keyboard handlers, and
Final Pass: Clean Build	100 / 100	native HTML5 dialog. Flawless zero-warning score across all 38 frontend components.

8.2 Key Architecture Refactors

- useGraphSimulation Hook:**
Extracted 350+ lines of physics loops and pan/zoom math into a reusable custom hook.
- HTML5 Native Dialog:** Converted CommandPalette into a keyboard-accessible <dialog> element.
- useReducer State Machine:**
Replaced tangled useState updates in PackagePage with FETCH_START, FETCH_SUCCESS, FETCH_ERROR actions.

9. Responsive Architecture & Pagination

CROSS-DEVICE POLISH

9.1 Responsive Layout Adaptations

The application implements fluid viewport scaling across mobile, tablet, and desktop:

- **Mobile Bottom Navigation:** Collapses 250px sidebar on ≤ 768 px screens into a fixed bottom navigation dock.
- **Fluid Typography:** Uses CSS clamp(1.75rem, 5vw, 2.75rem) to ensure headings never cause horizontal scroll.
- **Touch Target Sizing:** All buttons and chips meet the 44x44px minimum touch target standard with touch-action: manipulation.

9.2 Centered Bottom Pagination

The Package Registry Ledger features a clean pagination ribbon:

- **Default View:** Displays 10 items per page with clear Prev/Next and page number chips.
- **Custom Page Sizing:** Allows users to toggle 10, 20, 50, or 100 records per page.
- **Smart Reset:** Search or filter changes automatically reset the view to Page 1.

10. GitHub Repository & Setup Protocol

EXECUTION MANUAL

10.1 Repository Layout

```
cognodb-dependency-explorer/  
% % % backend/  
%   % % % src/  
%   %   % % % config/db.js           # Neo4j driver, keep-alive, retry wrapper  
%   %   % % % routes/packages.js      # Package CRUD & subgraphs  
%   %   % % % routes/queries.js       # Cypher console, Sandbox, Remediation  
%   %   % % % services/localGraphEngine.js # Zero-downtime graph fallback  
%   %   % % % server.js               # Express endpoint on port 4000  
%   %   % % % seed/seed.js            # CognoDB graph seed script  
% % % frontend/  
%   % % % src/  
%   %   % % % components/             # Modular visualizers, cards, navbars  
%   %   % % % pages/                  # Home, Package, Vulns, Graph, Cypher, Sandbox  
%   %   % % % api/client.js           # Centralized HTTP API client  
%   %   % % % index.html               # Monotone Ink HTML shell with favicon.svg  
%   %   % % % vite.config.js          # Vite build & backend proxy config  
% % % docs/                           # PDF Dossier, Markdown blueprints, HTML docs
```

10.2 Quick Start Commands

1. Clone & Configure Environment:

```
# In backend/.env:  
COGNODB_URI=bolt+s://db-6e111080.databases.cognodb.com  
COGNODB_USER=cognodb  
COGNODB_PASSWORD=your-secure-password  
PORT=4000
```

2. Seed Graph & Start Backend:

```
cd backend
npm install
npm run seed      # Populates 75 packages, 77 versions, 17 CVEs, 153 edges
npm start         # Runs Express on port 4000 (Connected to CognoDB)
```

3. Launch Frontend:

```
cd ../frontend
npm install
npm run dev      # Starts Vite dev server on http://localhost:5173
npm run build    # Compiles production bundle with 0 errors
```

11. Interview Defense & Video Script

11.1 Top Technical Interview Questions & Exact Defense Answers

- **Q1: Why is a graph database the right tool for this problem?:** Answer: Dependency exposure is inherently a graph problem of variable depth. SQL relational databases require expensive recursive CTEs that explode exponentially in memory past 2 hops. CognoDB uses index-free adjacency to traverse connections in $O(1)$ pointer operations.
- **Q2: How do you prevent Cypher injection and maximize performance?:** Answer: 100% of our queries are parameterized using \$name and \$rootName. This guarantees protection against injection and allows CognoDB to cache query execution plans.
- **Q3: How did you handle cloud database connection drops?:** Answer: We implemented maxConnectionLifetime: 30s in the neo4j-driver to recycle connections before cloud NAT firewalls drop them, added a 1-second pre-query liveness check, and built an exponential backoff withRetry wrapper.
- **Q4: How did you achieve 60 FPS graph rendering in React?:** Answer: We separated physics animation into a custom useGraphSimulation hook that mutates useRef pointers directly during requestAnimationFrame, completely bypassing React re-render cycles.

11.2 Video Walkthrough Script (2–3 Minutes)

- 0:00 – 0:30: Introduce project, CognoDB Bolt connectivity, and the software supply chain risk use case.
- 0:30 – 1:15: Search enterprise-api-gateway, demonstrate 4 tabs, shortest attack paths, and automated remediation.
- 1:15 – 1:45: Show maintainer blast radius leaderboard and searchable CVE catalog.
- 1:45 – 2:15: Open /cypher to execute live openCypher query, and /sandbox to simulate zero-day strike.
- 2:15 – 2:45: Conclude with 100/100 React Doctor score and production build.